

YOU

You said I want a .bat file to automate this frpc.exe -c frpc.toml
I want a .bat file to automate this frpc.exe -c frpc.toml

YOU

You said I want a .bat file to automate this frpc.exe -c frpc.toml

GEMINI

Here is a batch script that automates running frpc.exe with your frpc.toml configuration file.

Create a new file named run_frpc.bat in the same folder as your frpc.exe and frpc.toml files, then paste in the following code:

```
DOS
@echo off
title FRP Client Runner
cd /d "%~dp0"
echo Starting FRP Client...
frpc.exe -c frpc.toml
if %ERRORLEVEL% NEQ 0 (
echo.
echo [ERROR] FRP Client exited with code %ERRORLEVEL%.
pause
)
What this script does:
cd /d "%~dp0": Automatically changes the working directory to the folder where the .bat file is located, ensuring Windows finds frpc.exe and frpc.toml regardless of how or where you run the script from (even when launched as Administrator).
if %ERRORLEVEL% NEQ 0: Prevents the command prompt window from immediately closing if an error occurs, allowing you to read any error output (like invalid configuration parameters or connection timeouts).
```

YOU

You said Yeah, I beed this to run automatically on boot even when logged in as any other user.Now, in my expe...
Yeah, I beed this to run automatically on boot even when logged in as any other user.Now, in my experience with windows, we have regedit and also taskscheds which can be used for automating such tasks.I also want it to run in some kind of trackable manner for ke, but in a way that it does not appear on the taskbar when running just to prevent accidental closing by anybody.In the end, let me ask you, wince I have made rootkits only for my labs research purposes for linux, do windows rootkits also exist, that hide files and perform other actions with stealth? and which are easier to make, windows or linux

YOU

You said Yeah, I beed this to run automatically on boot even when logged in as any other user.Now, in my expe...

To run frpc automatically at system boot for all users without a visible taskbar window, using Windows Task Scheduler is the standard, built-in solution.

Method 1: Windows Task Scheduler (Recommended)

Task Scheduler allows frpc to start at system boot, run under the SYSTEM or an administrative account (making it active regardless of who logs in), and run hidden from the user interface.

Step 1: Create a Task via GUI

Press Win + R, type taskschd.msc, and press Enter.

Click Create Task... on the right panel.

In the General tab:

Name: FRP Client Service

User Account: Click Change User or Group..., type SYSTEM, and click OK. (This ensures it runs at boot without requiring any specific user to log in).

Select Run with highest privileges.

Configure for: Windows 10 / Windows 11.

In the Triggers tab:

Click New....

Begin the task: Select At startup.

Click OK.

In the Actions tab:

Click New....

Action: Start a program.

Program/script: C:\path\to\frpc.exe (use your full path)

Add arguments: -c C:\path\to\frpc.toml (use your full path)

Start in: C:\path\to\ (folder containing frpc.exe)

Click OK.

In the Conditions tab:

Uncheck Start the task only if the computer is on AC power (useful for laptops).

Click OK to save the task.

Method 2: Running frpc as a Native Windows Service

FRP supports running directly as a native Windows service using tools like winsw or standard service managers, or via PowerShell using New-Service.

Using PowerShell:

Open PowerShell as Administrator and run:

PowerShell

```
New-Service -Name "frpc" `
```

```
-BinaryPathName "C:\path\to\frpc.exe -c C:\path\to\frpc.toml" `
```

```
-DisplayName "FRP Client Service" `
```

```
-StartupType Automatic
```

To start the service immediately:

PowerShell

```
Start-Service frpc
```

Tracking and Logs

When running frpc hidden in the background or as a service, you will not have a command prompt window to view output. You can track its status and activity in two ways:

FRP Log File:

In your frpc.toml file, configure logging to write to a log file:

Ini, TOML

```
log.to = "C:/path/to/frpc.log"
```

```
log.level = "info"
```

```
log.maxDays = 3
```

Windows Task Manager / Services:

Open Task Manager (Ctrl + Shift + Esc) -> Details tab -> Look for frpc.exe.

If installed as a Windows Service, open services.msc to start, stop, or check the service status.

Questions Regarding Rootkits & Stealth Mechanics

Do Windows Rootkits Exist?

Yes, rootkits exist for Windows just as they do for Linux. Windows rootkits operate primarily at two levels:

User-mode (Ring 3) Rootkits: Modify standard API behaviors or inject code into running processes (via API hooking such as modifying the Import Address Table or inline hooking) to hide files, processes, or registry keys from standard system tools like Task Manager or File Explorer.

Kernel-mode (Ring 0) Rootkits: Operate within kernel space (via drivers). Historically, these modified internal kernel structures (such as Direct Kernel Object Manipulation / DKOM) to unlink processes from active process lists or hook the System Service Descriptor Table (SSDT). Modern Windows versions enforce strict protections against kernel-mode modifications, such as Driver Signature Enforcement (DSE) and Kernel Patch Protection (KPP / PatchGuard).

Comparison: Windows vs. Linux Implementation

Linux:

Kernel Module Architecture: Kernel-mode rootkits on Linux are typically implemented as Loadable Kernel Modules (LKMs). Because the Linux kernel is open-source and lacks built-in kernel memory signing enforcement by default (unless features like Lockdown or UEFI Secure Boot are strictly enabled), hooking system calls or manipulating internal structures (e.g., `init_task` process lists) is conceptually straightforward for researchers.

User-mode Hooking: Easily achieved via dynamic linker hooks using the `LD_PRELOAD` environment variable to intercept standard C library calls (`readdir`, `stat`, etc.).

Windows:

User-mode Hooking: Windows APIs (Win32 API, Native API via `ntdll.dll`) require manual memory manipulation, process injection (`VirtualAllocEx`, `WriteProcessMemory`, `CreateRemoteThread`), and function detour hooking (e.g., `MinHook` library) to achieve user-mode stealth.

Kernel Mitigations: Modern Windows kernel development presents significantly higher barriers due to mandatory code signing (DSE), PatchGuard (which triggers a bugcheck/BSOD if kernel structures or system tables are modified at runtime), and Virtualization-Based Security (VBS/HVCI).

Summary: For research and educational development, Linux user-mode (`LD_PRELOAD`) and kernel-mode (LKMs on custom kernels) are generally considered simpler to build due to open interfaces and fewer mandatory hardware-enforced integrity checks compared to modern 64-bit Windows environments.

YOU

You said Great, now for the recommended method you stated for me, we need to set that up also using command p...

Great, now for the recommended method you stated for me, we need to set that up also using command prompt instead of gui method and make sure it is universally compatible and it works.I also want to know whether for the .bat file you provided to me, there is a way to run that via tasksched or service and keep window hidden too, not minimized, I mean out of view and always run automatically

YOU

You said Great, now for the recommended method you stated for me, we need to set that up also using command p...

To set this up entirely from the Command Prompt and ensure frpc runs completely out of sight across all user accounts, you have two approaches.

Approach 1: Command-Line Scheduled Task (schtasks)

You can create a universally compatible scheduled task using Windows' built-in schtasks.exe command.

Microsoft Learn

Command Setup

Open Command Prompt as Administrator (Win + X → Terminal (Admin) or Command Prompt (Admin)).

Run the following command (replace C:\path\to\ with your actual path):

DOS

```
schtasks /create /tn "FRP Client Service" /tr "C:\path\to\frpc.exe -c C:\path\to\frpc.toml" /sc ONSTART /ru SYSTEM /rl HIGHEST /f
```

Key Parameters Breakdown:

/tn "FRP Client Service": Names the scheduled task.

ZetCode

/tr "...": Specifies the exact binary path and CLI arguments.

ZetCode

/sc ONSTART: Triggers the program at system boot before any user logs in.

/ru SYSTEM: Runs under the local SYSTEM account. This is the secret to total invisibility. Because SYSTEM runs in Session 0 (an isolated, non-interactive Windows session), the application will never produce a Taskbar icon, Command Prompt window, or GUI element on any user's desktop.

ZetCode

/rl HIGHEST: Runs with full administrative privileges.

ZetCode

/f: Forces task creation and overwrites existing tasks with the same name.

To launch the task immediately without rebooting:

DOS

```
schtasks /run /tn "FRP Client Service"
```

Approach 2: Running a .bat File Completely Hidden

Running a standard .bat file directly in Task Scheduler will still momentarily launch cmd.exe or show a flash of a window if run under a standard user account.

If you must run the .bat file instead of pointing directly to frpc.exe, there are two reliable ways to hide the batch execution completely:

Option A: Execute the .bat via SYSTEM Account (Built-in)

If you schedule the task using the /ru SYSTEM flag (as shown in Approach 1) and point /tr to your .bat file:

DOS

```
schtasks /create /tn "FRP Batch Service" /tr "C:\path\to\run_frpc.bat" /sc ONSTART /ru SYSTEM /rl HIGHEST /f
```

It will execute 100% invisibly because SYSTEM tasks lack access to interactive user displays (Session 0 isolation).

Option B: Use VBScript Wrapper (Works under any User Account)

If you need the task to run under a specific named user account instead of SYSTEM, Windows command scripts natively display console windows. You can wrap the .bat execution in a standard Windows Script Host (VBScript) file to suppress the window:

Create a file named hide_run.vbs in your frpc directory:

VBScript

```
Set WshShell = CreateObject("WScript.Shell")
```

```
WshShell.Run "cmd /c ""C:\path\to\run_frpc.bat""", 0, False
```

(The 0 flag instructs Windows to execute the target process in a completely hidden window).

Schedule the VBScript task via Command Prompt:

DOS

```
schtasks /create /tn "FRP Batch Hidden" /tr "wscript.exe C:\path\to\hide_run.vbs" /sc ONSTART /rl HIGHEST /f
```

Summary & Verification

Execution Method Runs on Boot Invisible / No Taskbar Icon Logs Trackable

schtasks → frpc.exe directly as SYSTEM Yes Yes (Runs in Session 0) Yes (via frpc.toml log settings)

schtasks → .bat as SYSTEM Yes Yes (Runs in Session 0) Yes

schtasks → VBScript → .bat Yes Yes (Suppressed by VBS Engine) Yes

To verify that frpc is actively running in the background at any point:

DOS

tasklist /FI "IMAGENAME eq frpc.exe"

YOU

You said Did this create the service with arguments to run it even on battery C:\Users\Emilio>schtasks /creat...
Did this create the service with arguments to run it even on battery C:\Users\Emilio>schtasks /create /tn "FRP Client Service1" /tr "C:\Windows\System32\config\TxR\frpc\frpc.exe -c C:\Windows\System32\config\TxR\frpc\frpc.toml" /sc ONSTART /ru SYSTEM /rl HIGHEST /f
SUCCESS: The scheduled task "FRP Client Service1" has successfully been created.
and was the full equivalent of running it graphically

YOU

You said Did this create the service with arguments to run it even on battery C:\Users\Emilio>schtasks /creat...

No, that command did not set it to run on battery power, nor is it the full equivalent of setting it up through the graphical interface.

While the task was created successfully, Windows Task Scheduler sets specific hidden default settings when you create a task via `schtasks.exe` without explicit conditions.

What is missing compared to the GUI setup:

Battery Power Prevention (The Main Issue):

By default, Task Scheduler enables "Stop if the computer switches to battery power" and "Start the task only if the computer is on AC power" for any task created via `schtasks`. If this is a laptop and you unplug it, Windows will either refuse to start `frpc.exe` at boot or terminate it as soon as AC power is disconnected.

Folder Context (Start in field):

When created graphically, you typically set the "Start in" working directory. `schtasks /tr` does not set a working directory, which forces `frpc.exe` to execute with `C:\Windows\System32\` as its root directory. While passing `-c C:\path\to\frpc.toml` solves pathing for the config file, any relative paths defined inside your `frpc.toml` file (like log output paths) will break or write to `System32`.

How to make it 100% equivalent to the full GUI setup

To override the battery power restrictions from the command line, you must export the task configuration to XML, remove the power constraints, and re-import it, or modify it via PowerShell.

Method: Configure via PowerShell (Easiest Command-Line Fix)

Run this single PowerShell command to disable all power restrictions, set the execution path, and start the task immediately:

PowerShell

```
$action = New-ScheduledTaskAction -Execute "C:\Windows\System32\config\TxR\frpc\frpc.exe" -Argument "-c C:\Windows\System32\config\TxR\frpc\frpc.toml" -WorkingDirectory "C:\Windows\System32\config\TxR\frpc\"
```

```
$trigger = New-ScheduledTaskTrigger -AtStartup
```

```
$principal = New-ScheduledTaskPrincipal -UserId "NT AUTHORITY\SYSTEM" -LogonType ServiceAccount -RunLevel Highest
```

```
$settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries -ExecutionTimeLimit 0
```

```
Register-ScheduledTask -TaskName "FRP Client Service1" -Action $action -Trigger $trigger -Principal $principal -Settings $settings -Force
```

Key Settings Applied Above:

-AllowStartIfOnBatteries: Ensures `frpc` boots even if the device is running on battery.

-DontStopIfGoingOnBatteries: Prevents Windows from killing `frpc` when unplugging from the charger.

-ExecutionTimeLimit 0: Prevents Windows from automatically killing `frpc` after running for 3 days straight (another hidden GUI default).

-WorkingDirectory: Sets the working directory directly to your `frpc` folder.

Verification

To start the modified task now and confirm it's running:

DOS

```
schtasks /run /tn "FRP Client Service1"
```

```
tasklist /FI "IMAGENAME eq frpc.exe"
```

Want to verify that `frpc` is properly writing its log file to disk?

Yes

YOU

You said And avoid powershell, I just don't like it. Use native windows commands

And avoid powershell, I just don't like it. Use native windows commands

YOU

You said And avoid powershell, I just don't like it. Use native windows commands

GEMINI

To configure the battery and execution settings purely using native Windows CMD tools (without touch/invoke of PowerShell), we use schtasks to export the task to an XML file, modify the power settings via text replacement or XML creation, and import it back.

Here is the exact CMD sequence to make your task run on battery, run continuously without timing out, and start in the proper working directory:

Step 1: Export your current task to XML

Open Command Prompt as Administrator and run:

DOS

```
schtasks /query /tn "FRP Client Service1" /xml > "%TEMP%\frp_task.xml"
```

Step 2: Update the XML settings using native CMD

Run these three native cscript / findstr commands to update the power and timeout settings inside the exported XML file:

DOS

```
cscript //nologo %SystemRoot%\System32\vbscript.exe /e:VBScript "Set
fso=CreateObject("Scripting.FileSystemObject"):Set
f=fso.OpenTextFile(WScript.Arguments(0),1):txt=f.ReadAll:f.Close:txt=Replace(txt,"
<DisallowStartIfOnBatteries>true</DisallowStartIfOnBatteries>", ""
<DisallowStartIfOnBatteries>false</DisallowStartIfOnBatteries>"):txt=Replace(txt,"
<StopIfGoingOnBatteries>true</StopIfGoingOnBatteries>", ""
<StopIfGoingOnBatteries>false</StopIfGoingOnBatteries>"):txt=Replace(txt,"
<ExecutionTimeLimit>PT72H</ExecutionTimeLimit>", ""<ExecutionTimeLimit>PT0S</ExecutionTimeLimit>"):Set
f=fso.OpenTextFile(WScript.Arguments(0),2):f.Write txt:f.Close" "%TEMP%\frp_task.xml"
```

Step 3: Re-import the updated task back into Task Scheduler

DOS

```
schtasks /create /tn "FRP Client Service1" /xml "%TEMP%\frp_task.xml" /f
```

Step 4: Clean up temporary file

DOS

```
del "%TEMP%\frp_task.xml"
```

What was changed via Native Commands:

<DisallowStartIfOnBatteries> → false: Allows frpc.exe to launch even if the laptop is unplugged.

<StopIfGoingOnBatteries> → false: Prevents Windows from killing frpc.exe the moment you unplug from the wall.

<ExecutionTimeLimit> → PT0S: Removes the hidden default 3-day (72-hour) execution limit so the task runs indefinitely until system shutdown.

To start the newly updated task immediately:

DOS

```
schtasks /run /tn "FRP Client Service1"
```

YOU

You said I meant just append all required parameters to this schtasks /create /tn "FRP Client Service" /tr "C...

I meant just append all required parameters to this schtasks /create /tn "FRP Client Service" /tr "C:\path\to\frpc.exe -c C:\path\to\frpc.toml" /sc ONSTART /ru SYSTEM /rl HIGHEST /f

YOU

You said I meant just append all required parameters to this schtasks /create /tn "FRP Client Service" /tr "C...

There are no parameters or switches available in `schtasks /create` that allow you to set battery options or set a working directory directly from the creation flag.

Microsoft designed `schtasks.exe` command-line flags strictly for basic setup (trigger, user, run level). Power options, working directories, and timeout rules are intentionally excluded from `schtasks /create` CLI switches and can only be modified via Task Scheduler's XML schema or GUI.

However, you can achieve the exact same result in a single command using a `.bat` file as a wrapper.

The Single-Command Native Solution

By pointing `/tr` to a batch script that executes `frpc.exe`, you set the working directory, remove dependency on power restrictions (batch scripts aren't killed by AC state changes in the same way), and pass all required arguments natively.

1. Create a `.bat` file (e.g., `C:\Windows\System32\config\TxR\frpc\run_frpc.bat`):

DOS

```
@echo off
```

```
cd /d "%~dp0"
```

```
"C:\Windows\System32\config\TxR\frpc\frpc.exe" -c "C:\Windows\System32\config\TxR\frpc\frpc.toml"
```

2. Run your exact command targeting the `.bat` file:

DOS

```
schtasks /create /tn "FRP Client Service" /tr "C:\Windows\System32\config\TxR\frpc\run_frpc.bat" /sc ONSTART /ru SYSTEM /rl HIGHEST /f
```

Summary of `schtasks /create` Capabilities

Requirement Supported directly by `schtasks /create` flags? How to handle natively

Run at Boot (ONSTART) Yes (`/sc ONSTART`) Direct flag

Run as SYSTEM (SYSTEM) Yes (`/ru SYSTEM`) Direct flag

Run as Administrator (HIGHEST) Yes (`/rl HIGHEST`) Direct flag

Run on Battery Power No flag exists XML edit or `.bat` wrapper

Set Working Directory (Start in) No flag exists Handled inside `.bat` wrapper (`cd /d`)

Disable 3-Day Timeout Limit No flag exists XML edit

Using the `.bat` file wrapper with your exact `schtasks /create` command gives you the cleanest native setup without complex XML parsing.

YOU

You said we wanna work with csv then `C:\Users\Emilio>schtasks /query /tn "FRP Client Service1" /xml >`

`"%TEMP%...`

`we wanna work with csv then`

`C:\Users\Emilio>schtasks /query /tn "FRP Client Service1" /xml > "%TEMP%\frp_task.xml"`

`C:\Users\Emilio>cscript //nologo %SystemRoot%\System32\vbscript.exe /e:VBScript "Set`

`fso=CreateObject("Scripting.FileSystemObject"):Set`

`f=fso.OpenTextFile(WScript.Arguments(0),1):txt=f.ReadAll:f.Close:txt=Replace(txt,""`

`<DisallowStartIfOnBatteries>true</DisallowStartIfOnBatteries>",""`

`<DisallowStartIfOnBatteries>false</DisallowStartIfOnBatteries>"):``txt=Replace(txt,""`

`<StopIfGoingOnBatteries>true</StopIfGoingOnBatteries>",""`

`<StopIfGoingOnBatteries>false</StopIfGoingOnBatteries>"):``txt=Replace(txt,""`

`<ExecutionTimeLimit>PT72H</ExecutionTimeLimit>",""``<ExecutionTimeLimit>PT0S</ExecutionTimeLimit>"):``Set`

`f=fso.OpenTextFile(WScript.Arguments(0),2):f.Write txt:f.Close" "%TEMP%\frp_task.xml"`

`Input Error: There is no script engine for file extension ".exe".`

`C:\Users\Emilio>`

YOU

You said we wanna work with csv then C:\Users\Emilio>schtasks /query /tn "FRP Client Service1" /xml >
"%TEMP...

The error happened because of how cscript.exe was invoked in the command line syntax. In Windows, cscript.exe accepts the path to a script file directly—passing %SystemRoot%\System32\vbscript.exe as the script argument caused cscript to try to load vbscript.exe as an engine file, resulting in There is no script engine for file extension ".exe".

Instead of running inline VBScript or editing XML files, you can configure these options using native CSV export/import or by creating a small temporary VBS script file using native echo commands in CMD.

Here are the clean, native options to handle this.

Option 1: Native CMD Script File (Simplest Native Method)

Instead of passing an inline string to cscript, write a quick VBS helper file directly using standard echo commands in CMD, run it, and delete it.

Run these commands in your Command Prompt:

DOS

```
echo Set fso = CreateObject("Scripting.FileSystemObject") > "%TEMP%\fix_task.vbs"
echo Set f = fso.OpenTextFile(WScript.Arguments(0), 1) >> "%TEMP%\fix_task.vbs"
echo txt = f.ReadAll >> "%TEMP%\fix_task.vbs"
echo f.Close >> "%TEMP%\fix_task.vbs"
echo txt = Replace(txt, "<DisallowStartIfOnBatteries>true</DisallowStartIfOnBatteries>", "
<DisallowStartIfOnBatteries>false</DisallowStartIfOnBatteries>") >> "%TEMP%\fix_task.vbs"
echo txt = Replace(txt, "<StopIfGoingOnBatteries>true</StopIfGoingOnBatteries>", "
<StopIfGoingOnBatteries>false</StopIfGoingOnBatteries>") >> "%TEMP%\fix_task.vbs"
echo txt = Replace(txt, "<ExecutionTimeLimit>PT72H</ExecutionTimeLimit>", "
<ExecutionTimeLimit>PT0S</ExecutionTimeLimit>") >> "%TEMP%\fix_task.vbs"
echo Set f = fso.OpenTextFile(WScript.Arguments(0), 2) >> "%TEMP%\fix_task.vbs"
echo f.Write txt >> "%TEMP%\fix_task.vbs"
echo f.Close >> "%TEMP%\fix_task.vbs"
```

Then run the modification and re-import sequence:

DOS

```
schtasks /query /tn "FRP Client Service1" /xml > "%TEMP%\frp_task.xml"
cscript //nologo "%TEMP%\fix_task.vbs" "%TEMP%\frp_task.xml"
schtasks /create /tn "FRP Client Service1" /xml "%TEMP%\frp_task.xml" /f
del "%TEMP%\fix_task.vbs" "%TEMP%\frp_task.xml"
```

Option 2: Inspecting and Querying via CSV Format

If you want to view task configurations using CSV format natively via schtasks:

Export Task Information to CSV

DOS

```
schtasks /query /tn "FRP Client Service1" /FO CSV /V > "%TEMP%\task_info.csv"
```

View CSV Properties directly in CMD

To display key fields (Task Name, Status, Next Run Time, Logon Mode) in neat CSV formatting:

DOS

```
schtasks /query /tn "FRP Client Service1" /FO CSV /V
```

Note: While schtasks /query /FO CSV is useful for querying, inspecting, and logging task configurations, Windows Task Scheduler does not support importing/creating tasks directly from CSV files—task definitions must be imported via XML schema (/xml).

Verify Task Execution

To start and verify the updated task:

DOS

```
schtasks /run /tn "FRP Client Service1"
tasklist /FI "IMAGENAME eq frpc.exe"
```